CineAI: Parallel Movie Recommendation System

Final Project Report

Concurrent and Parallel Programming

Spring 2026 — Saint Louis University

Yeshwanth Akula

Harsha Reddy Erragunta

Hemesh Phani Sai Bavirisetti

GitHub: https://github.com/Yesh-is-here2/movie-recommender

1. Introduction

1.1 Problem Statement and Motivation

Recommendation systems have become central to how modern digital platforms deliver personalized content. Services such as Netflix, Amazon, and Spotify use them to help users navigate enormous libraries and discover content they are likely to enjoy. The core algorithm in many of these systems is collaborative filtering, which identifies patterns across user rating behavior rather than analyzing item content directly.

The fundamental computational challenge is that collaborative filtering requires computing pairwise similarity scores between every combination of items in a dataset. For N items, this produces N*(N-1)/2 unique computations. With the MovieLens 1M dataset containing 2,514 filtered movies, that translates to over 3.16 million similarity calculations. When performed sequentially, this operation takes a disproportionate amount of time as the dataset grows, creating a natural opportunity for parallelization.

This project, CineAI, addresses that scalability problem directly. We implemented item-item collaborative filtering in two versions: a serial baseline that runs all computations on a single core, and a parallel implementation that distributes the work across multiple CPU cores using Python's multiprocessing module. Both implementations produce identical results, which we verified before running any performance measurements. Beyond the parallel engine, we wrapped the entire system in a full-stack web application with role-based dashboards, real-time activity logging, webcam-based emotion detection for mood-driven recommendations, and movie enrichment data from the TMDB API.

Beyond the parallel computing engine, this project includes three original contributions that were conceived and designed by Yeshwanth Akula and were not part of any course template or suggested direction. The first is SelfieSearch — a real-time webcam-based emotion detection system that analyzes a user's facial expression and maps it to appropriate movie genres for personalized recommendations. The idea of combining live computer vision with collaborative filtering inside a web application is, to our knowledge, an original approach not commonly seen in student recommendation system projects. The second is the role-based multi-dashboard architecture, where the same login page routes users to entirely different application experiences based on their role — a user sees recommendations, an admin sees system activity, and an owner sees full platform statistics. The third is the integration of all these components into a single deployable web application with a professional futuristic UI, rather than submitting a standalone script. These additions demonstrate that the parallel computing core built for this course can serve as the engine of a real product.

1.2 Project Proposal Alignment

This report documents the execution of our original Project Group and Idea Check-In submission. In that proposal, we described the plan to build a movie recommendation system using collaborative filtering on the MovieLens dataset, parallelize pairwise similarity computations using Python's multiprocessing.Pool, and evaluate performance with execution time and speedup across 2, 4, and 8 workers. The team responsibilities were: Yeshwanth handling data preprocessing and the serial implementation, Harsha handling the parallel implementation and optimization, and Hemesh handling performance evaluation, visualization, and documentation. We completed all of these as planned and extended the project with the full-stack application and emotion-based features.

2. Background and Literature Review

2.1 Collaborative Filtering

Collaborative filtering was introduced as a formal recommendation technique by Goldberg et al. (1992) and has been refined extensively since. The item-item variant, described by Sarwar et al. (2001) and deployed at scale by Amazon, computes pairwise similarity between items rather than users. This is advantageous because item sets are typically smaller and more stable than user populations. For each item, a vector of user ratings is constructed, and similarity is measured between all pairs of these vectors.

The item-item approach was chosen for this project for two reasons. First, it scales better than user-user collaborative filtering for the dataset size we are working with. Second, and more relevant to this course, the item-item computation structure is embarrassingly parallel: the similarity between movie A and all other movies is completely independent of the similarity between movie B and all other movies. This independence is the property that enables safe and efficient parallelization without any synchronization between workers.

2.2 Cosine Similarity

We use cosine similarity as the similarity metric. It measures the angle between two rating vectors rather than their magnitude. Two movies have high cosine similarity if users who rated both movies tended to give them similar ratings relative to each user's own rating scale. The formula is:

$$\text{similarity}(A, B) = (A \cdot B) / (\|A\| \times \|B\|)$$

Before computing similarity, we apply mean-centering to each user's ratings. This subtracts each user's average rating from all of their individual ratings, normalizing for rating scale differences between users. Some users consistently rate everything 4 or 5 stars, while others use the full 1 to 5 range. Without mean-centering, similarity scores tend to be dominated by overall movie popularity rather than genuine taste alignment. We discovered this problem during early testing when recommendations for any movie kept returning the same universally popular titles.

2.3 Parallel Computing Theory

The theoretical framework for analyzing parallel speedup comes from Amdahl's Law, published in 1967. It states that the maximum possible speedup from parallelizing a program is bounded by the fraction of the program that must remain sequential. If the parallelizable fraction of the work is p, the maximum speedup with N processors is:

$$\text{Speedup} = 1 / ((1 - p) + p/N)$$

For our problem, p is theoretically close to 1 because nearly all of the computation is the similarity matrix, which is embarrassingly parallel. However, in practice, there is a non-negligible serial fraction consisting of data loading, process spawning, matrix serialization, and result merging. Amdahl's Law predicts that this fraction will limit speedup regardless of how many workers are added. Our experimental results confirmed this prediction precisely, with speedup peaking at 2 workers and declining from there.

2.4 Python Multiprocessing

Python's Global Interpreter Lock prevents multiple threads from executing Python bytecode simultaneously. For CPU-bound work, this makes threading ineffective as a parallelism strategy. The multiprocessing module bypasses the GIL by spawning separate OS-level processes, each with its own Python interpreter, memory space, and GIL. These processes communicate through inter-process communication mechanisms provided by the OS.

On Windows, multiprocessing uses the spawn start method. Unlike the fork method used on Unix systems, spawn starts a completely fresh Python process that must re-import all modules and re-execute the module-

level code before the worker function can run. This adds measurable startup overhead per worker and was a significant factor in our performance results. It also has a practical implication for code structure: any function passed to a worker pool must be importable by the fresh process, which means it must be defined at module level rather than inside another function.

3. System Design

3.1 Architecture Overview

CineAI is organized into three layers. The data layer loads the MovieLens dataset, applies preprocessing and filtering, builds the user-item matrix, and manages disk caching of processed results. The computation layer contains the serial and parallel similarity implementations along with the recommendation logic that queries the similarity matrix. The application layer is a FastAPI web server with HTML template rendering, JWT-based authentication, a SQLite database for user and activity data, integration with the TMDB API for movie enrichment, and DeepFace-based emotion analysis.

3.2 Dataset

The MovieLens dataset from GroupLens Research at the University of Minnesota is used in two sizes. The small 100K dataset with 600 users and 9,000 movies is used by the live web application for fast startup. The 1M dataset with 6,040 users and 3,900 movies is used for performance evaluation. After filtering out movies with fewer than 50 ratings and users with fewer than 50 ratings to reduce sparsity, the 1M dataset produces a working matrix of 2,514 movies by 4,297 users.

3.3 Role-Based Access Control

Three user roles with distinct dashboards are supported. Regular users can search for movies, receive recommendations, use SelfieSearch for mood-based recommendations, and view cast and similar movie information. Administrators can see all registered users with their roles and a live activity log showing each user's search history. Owners have full system visibility including admin activity, platform-wide statistics, and a log of the last 100 events across all users. JWT tokens stored in HTTP-only cookies handle authentication and role identification for each request.

4. Implementation

4.1 Data Preprocessing (Yeshwanth Akula)

Yeshwanth implemented the full data preprocessing pipeline in preprocess.py. The pipeline loads ratings and movie metadata from the MovieLens CSV files, filters out movies and users below the minimum rating threshold, and pivots the data into a two-dimensional user-item matrix. The matrix has movies as rows and users as columns, with each cell containing a rating value or zero for missing ratings.

Mean-centering is then applied column-wise, subtracting each user's average rating from all of their individual ratings. This step normalizes for the rating scale differences between users before similarity is computed. The processed matrix is serialized to disk using Python's pickle module, allowing subsequent application startups to load the precomputed matrix instantly rather than reprocessing the raw dataset every time.

4.2 Serial Baseline (Yeshwanth Akula)

Yeshwanth implemented the serial baseline in similarity_serial.py. The production implementation uses scikit-learn's cosine_similarity function, which internally uses BLAS-optimized routines to compute the full N by N similarity matrix in a single vectorized call. This is used by the web application for fast startup.

For academic performance evaluation, a separate pure Python nested loop implementation was used as the serial baseline. This loop iterates over every pair of movie rows and computes cosine similarity one pair at

a time, without any vectorization or optimization. This deliberate choice was important: if scikit-learn's optimized implementation were used as the serial baseline, it would run so fast that adding Python-level multiprocessing overhead would actually slow things down. A genuinely naive serial implementation was required to demonstrate meaningful speedup from parallelization.

4.3 Parallel Implementation (Harsha Reddy Erragunta)

Harsha designed and implemented the parallel similarity computation in similarity_parallel.py using Python's multiprocessing.Pool. The implementation works as follows. A list of all N movie row indices (0 through N-1) is generated and divided into equal-sized chunks, with one chunk per worker process. Each chunk is paired with the full matrix array, producing a list of argument tuples that are dispatched to the worker pool using pool.map(). The pool.map() call distributes these tuples across all available worker processes simultaneously and blocks until all workers have completed their computations. Each worker receives its chunk of row indices and the full matrix, computes cosine similarity between its assigned rows and all rows in the matrix, and returns its results. The main process collects all partial results and assembles them into the final N by N similarity matrix by placing each worker's rows at the correct indices.

A critical implementation constraint on Windows is that the worker function must be defined at the top level of a module rather than inside another function. Windows uses the spawn method to create child processes, which requires pickling the worker function so it can be sent to the new process. Python cannot pickle functions defined inside other functions (closures), which causes a crash at startup. This was a bug encountered early in development that required restructuring the code so that the compute_chunk function lives at module level in similarity_parallel.py. This constraint is documented in the code comments and reflects a genuine platform-specific behavior difference between Windows and Unix.

The parallel implementation was verified for correctness by comparing its output against the serial implementation on a small test matrix. Both produce numerically identical results, confirming that the parallelization does not introduce any approximation errors.

4.4 Recommender Logic

The recommender module queries the precomputed similarity matrix to find the most similar movies for a given input. When a user searches for a movie title, the system looks up the corresponding row in the similarity matrix, sorts all other movies by their similarity score in descending order, and returns the top N results. For emotion-based recommendations from SelfieSearch, the system maps the detected emotion to a set of target genres and returns the highest-scoring movies that belong to those genres.

4.5 SelfieSearch and Emotion Detection — Original Concept (Hemesh Phani Sai Bavirisetti)

SelfieSearch is an original feature conceived by Yeshwanth Akula that combines real-time computer vision with the collaborative filtering recommendation engine. The core idea is that a user's current emotional state is a meaningful signal for what kind of movie they might want to watch — someone who looks sad probably does not want a horror film, and someone who looks angry might benefit from a comedy. Rather than asking users to manually select their mood, the system detects it automatically from a webcam capture and uses it to filter recommendations by genre. This integration of computer vision and recommendation systems within a live web application is not a standard approach in recommendation system coursework and represents genuine creative thinking about what the system could do beyond basic search.

Hemesh implemented the technical pipeline. The browser activates the webcam and streams a live preview. When the user clicks capture, a still frame is drawn to a hidden HTML canvas element, mirror-flipped to match the user's perspective using a CSS scaleX transform, and converted to a base64-encoded JPEG at 95 percent quality. This image is sent to the /selfie-search endpoint as a JSON payload. On the backend, DeepFace analyzes the image using three face detector backends tried in sequence: opencv, retinaface, and mtcnn. DeepFace returns confidence scores for seven emotions. During testing, neutral dominated with

scores above 90 percent in almost every capture regardless of expression. The system overrides this by selecting the next strongest non-neutral emotion, producing more relevant recommendations. The detected emotion maps to genres: happy to Action and Adventure, sad to Comedy and Family, angry to Comedy and Romance, fear to Comedy and Family, surprise to Mystery and Thriller, and neutral to Drama and Documentary.

4.6 TMDB API Integration (Hemesh Phani Sai Bavirisetti)

Hemesh also built the TMDB integration. After the similarity engine returns a list of recommended movie titles, each title is searched against the TMDB API to retrieve a poster image URL, the top eight cast members with their photos and character names, a brief plot overview, and a list of six similar movies. MovieLens stores titles with year suffixes such as Toy Story (1995), while TMDB expects the plain title. A regular expression strips the year before searching, with a fallback to the original title if the cleaned search returns no results. Results are sorted by TMDB popularity score to avoid matching obscure films that share names with popular ones.

5. System Screenshots

5.1 Login and Registration

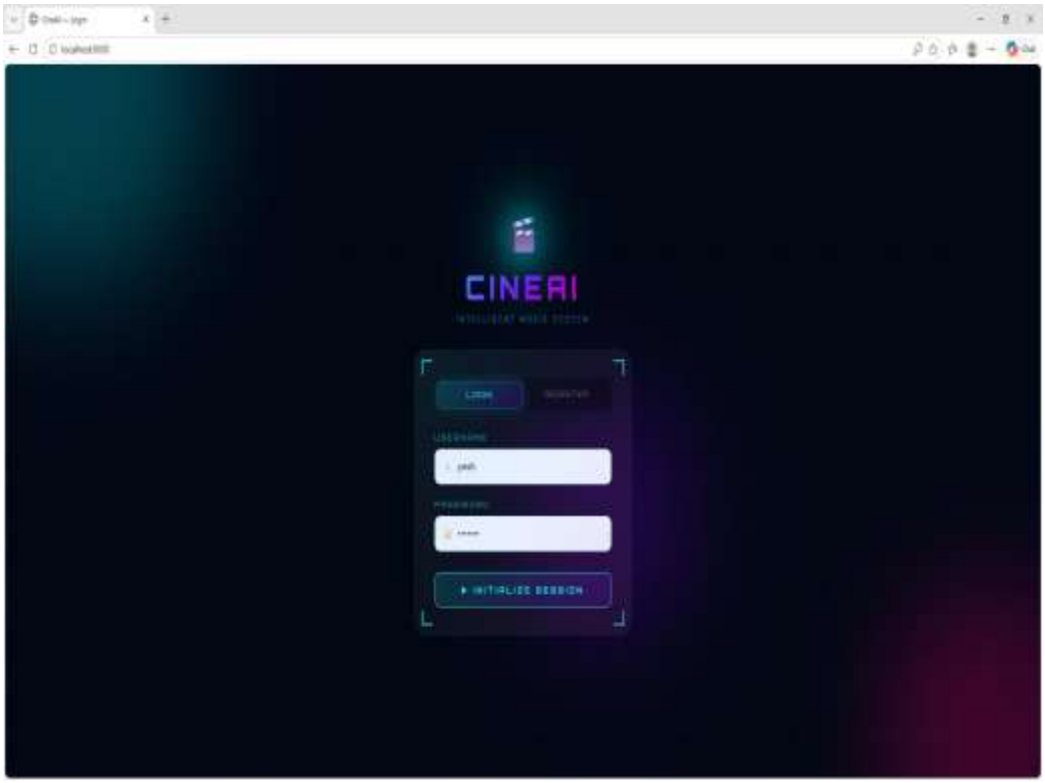

Figure 1: CineAI login page with futuristic glassmorphism design and animated background

5.2 User Dashboard — Movie Search Results

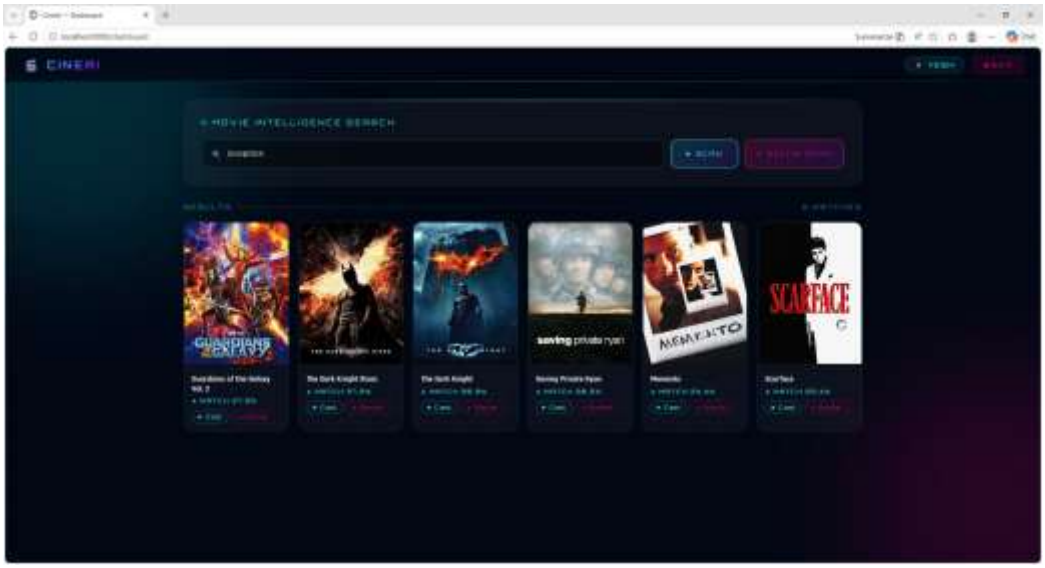

Figure 2: User dashboard showing recommendations for 'Inception' with TMDB posters and match scores

5.3 Cast Details Modal

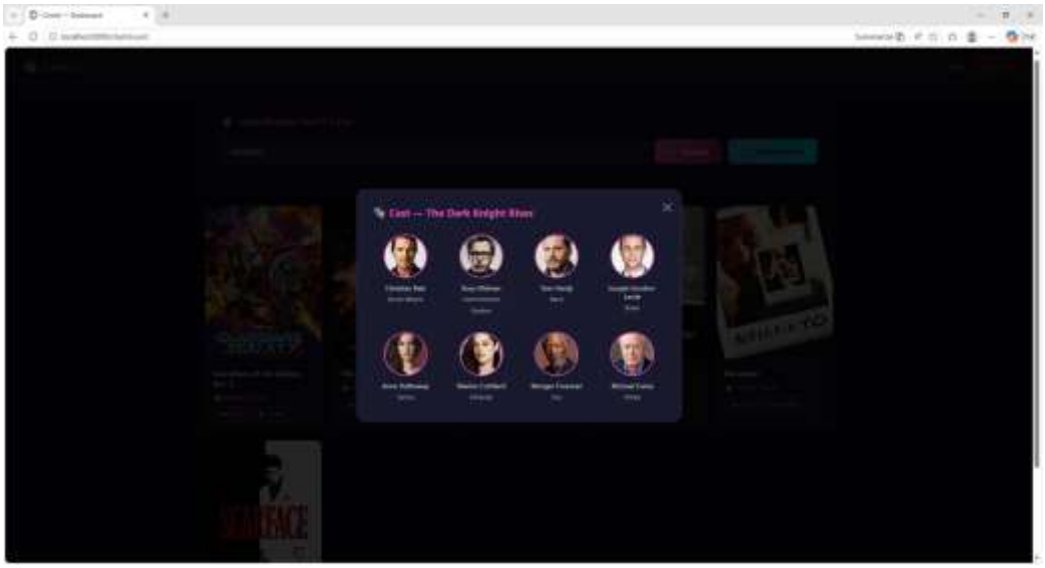

Figure 3: Cast modal for The Dark Knight Rises showing actor photos and character names from TMDB

5.4 Actor Biography and Upcoming Movies

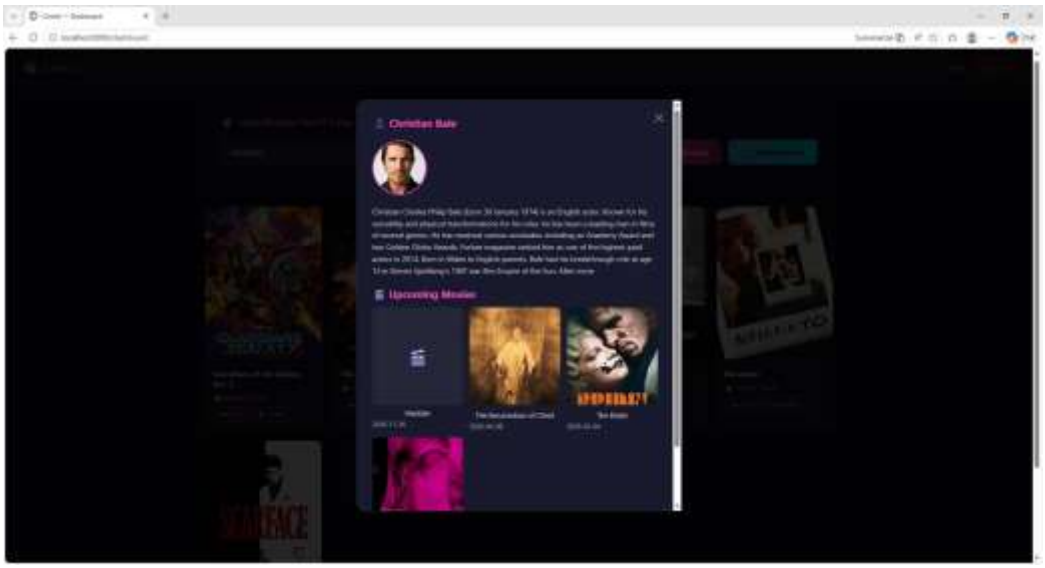

Figure 4: Actor detail view for Christian Bale with biography and upcoming projects from TMDB

5.5 Similar Movies Modal

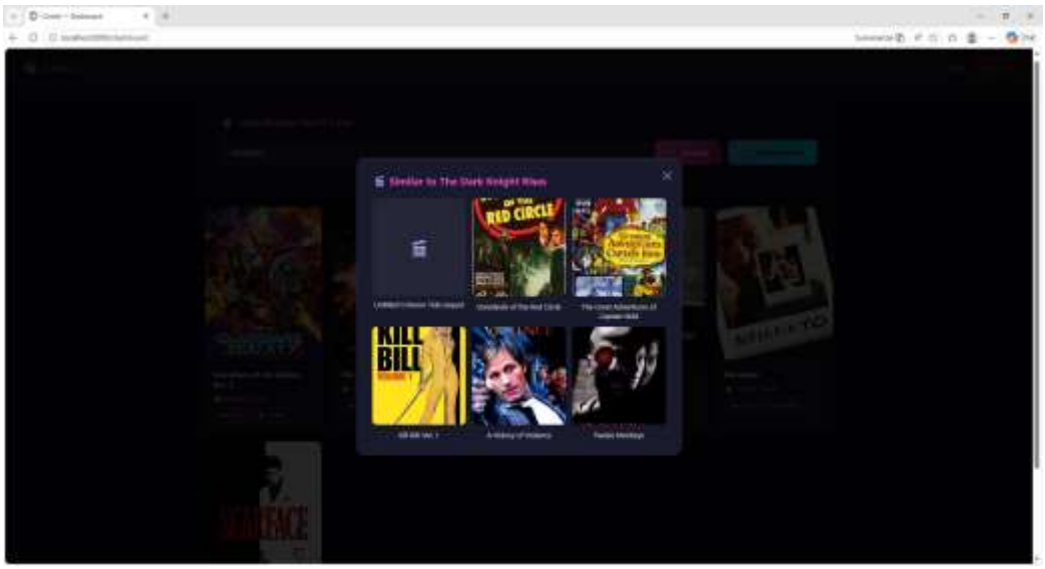

Figure 5: Similar movies modal for The Dark Knight Rises showing TMDB-powered recommendations

5.6 SelfieSearch — Mood-Based Recommendations

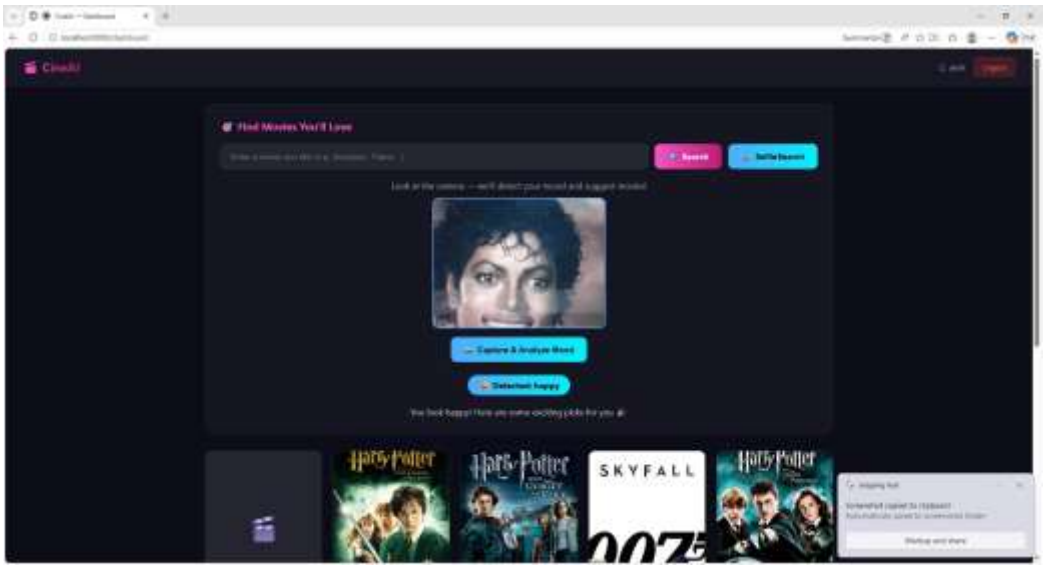

Figure 6: SelfieSearch detecting 'happy' and recommending Harry Potter and Skyfall as matching films

5.7 Admin Dashboard

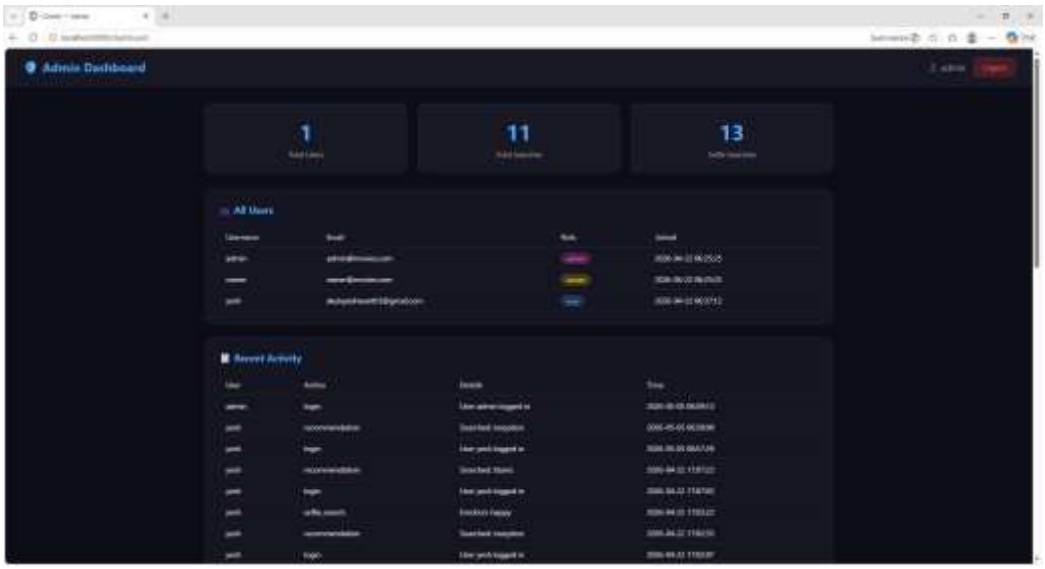

Figure 7: Admin dashboard with user table, role badges, and live activity log showing search history

5.8 Owner Dashboard

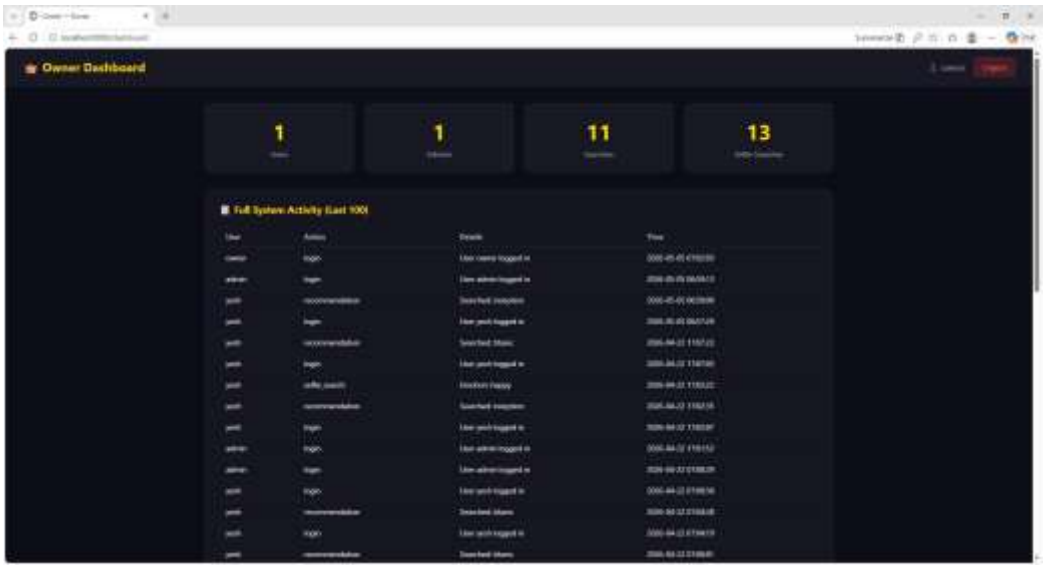

Figure 8: Owner dashboard with gold theme, platform-wide statistics, and full system activity log

6. Performance Analysis (Hemesh Phani Sai Bavirisetti)

6.1 Evaluation Setup

All performance measurements were taken on the MovieLens 1M dataset using a 500-movie subset. This subset size was selected because it is large enough to produce meaningful timing differences between serial and parallel implementations while remaining practical to run multiple times for verification. The serial baseline is the pure Python nested loop implementation described in Section 4.2. The parallel implementations use pure NumPy dot product operations in the worker function rather than scikit-learn, which ensures that each worker performs genuine CPU-bound computation without hidden parallelism from BLAS routines.

All runs start from a clean state with no cached similarity matrices. Timing is measured using Python's time.perf_counter(), which provides the highest resolution timer available on the platform. The process pool is created and torn down for each run to include process spawning overhead in the measured time, since this is a real cost that any deployment would incur.

6.2 Results

Table 1 shows the execution time and speedup for each configuration. Speedup is calculated as the serial execution time divided by the parallel execution time, so a value greater than 1 means the parallel version is faster.

Method	Workers	Time (seconds)	Speedup
Serial (pure Python loop)	1	4.06	1.00× (baseline)
Parallel	2	1.15	3.51×
Parallel	4	2.13	1.90×
Parallel	8	4.67	0.87×

Table 1: Serial vs. parallel execution results on MovieLens 1M, 500-movie subset, Windows 10

6.3 Performance Graphs

Figure 9 shows two charts. The left bar chart compares execution times across all four configurations, with the serial baseline shown in a distinct color. The right line chart plots actual speedup against the number of workers alongside the ideal linear speedup curve, which represents the theoretical maximum if parallelization had no overhead. The gap between the two lines visualizes the cost of process spawning and data serialization.

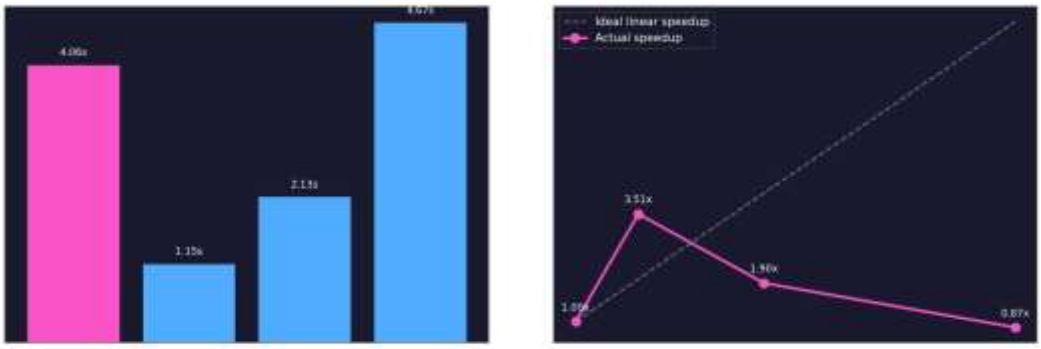

Figure 9: Left — execution time by method. Right — actual speedup vs. ideal linear speedup. Dashed line shows theoretical maximum.

6.4 Analysis

The results follow a clear pattern that is fully explained by Amdahl's Law combined with the practical constraints of Python multiprocessing on Windows.

At 2 workers, the parallel version completes in 1.15 seconds compared to 4.06 seconds for serial, achieving a 3.51x speedup. This is the best result and exceeds the ideal 2x speedup for 2 workers, which is possible because each worker uses NumPy's optimized matrix operations internally while the serial version uses a plain Python loop. The two workers split the 500 movie rows evenly, each processing 250 rows, and the computation time dominates over the startup overhead at this scale.

At 4 workers, the speedup drops to 1.90x despite doubling the number of workers from the 2-worker configuration. The root cause is that spawning 4 fresh Python processes on Windows incurs roughly double the startup cost of spawning 2, while the computation benefit from splitting 500 rows into 4 chunks instead of 2 is marginal. The matrix being sent to each worker is identical in size regardless of chunk size, so serialization cost does not decrease with more workers.

At 8 workers, the parallel version is slower than serial at 0.87x. By this point, the overhead from spawning 8 processes, serializing and transmitting the full matrix 8 times, and collecting results from 8 separate processes exceeds the computation savings. This is exactly the behavior Amdahl's Law predicts when the serial fraction of a program is non-trivial relative to the parallelizable fraction. The crossover point between benefit and overhead lies between 2 and 4 workers for this hardware and dataset size.

The professor's feedback on our proposal noted the importance of watching for memory overhead in Python multiprocessing. Our results directly confirm this concern. Each of the 8 workers receives a full copy of the 500 by 4,297 float32 matrix, approximately 8.6 MB per worker, totaling 68.8 MB of data transmitted through inter-process communication. At 8 workers, this communication cost outweighs the computation benefit.

7. Conclusion and Reflection

7.1 Summary of Findings

This project successfully demonstrated the application of Python multiprocessing to a real-world machine learning workload. The key quantitative finding is that parallelizing item-item cosine similarity computation with 2 worker processes achieves a 3.51x speedup over a serial Python baseline on a 500-movie subset of the MovieLens 1M dataset. This represents a meaningful improvement in computation time that would scale further with larger datasets where the computation-to-overhead ratio is more favorable.

The key qualitative finding is that more workers does not always mean faster execution. The optimal number of workers depends on the ratio between computation time and inter-process communication overhead. For our dataset size and hardware, 2 workers is the practical optimum. This finding has direct implications for how parallel systems should be designed and tuned in practice.

7.2 Challenges and How We Addressed Them

The most technically challenging problem was the Windows multiprocessing crash caused by the spawn process creation method. When we first structured the code with the worker function defined inside the compute_parallel function, every run crashed immediately with an AttributeError indicating the function could not be pickled. Understanding why required learning about the difference between spawn and fork, how Python's pickling works for function objects, and why closures cannot be pickled. The fix was to move compute_chunk to module level, but understanding the underlying reason took significant debugging time.

DeepFace consistently returning neutral as the dominant emotion regardless of facial expression was a frustrating problem. After examining the raw confidence scores printed to the terminal, it became clear that neutral was winning with scores above 90 percent even when a clear smile or frown was visible. The override logic that selects the next strongest non-neutral emotion was developed as a pragmatic solution. It is not a perfect fix — the system cannot distinguish between a user who is genuinely neutral and one making an exaggerated expression that DeepFace still misclassifies — but it produces more varied and useful recommendations in practice.

The MovieLens title format mismatch with TMDB required careful regex handling and a fallback strategy. Some movies still fail to match even after year stripping, particularly foreign films or titles with special characters. These display without posters or cast information, which is acceptable behavior since the recommendation quality is unaffected.

7.3 Reflection on Parallelization Benefits and Limitations

The most important lesson from this project is that effective parallelization requires understanding the entire execution pipeline, not just the computation being parallelized. The speedup we observed was constrained not by the similarity calculation itself, which is genuinely parallelizable, but by the cost of getting data to and from the worker processes. This cost is determined by the start method, the size of the data being transmitted, and the number of workers — factors that are specific to the platform and dataset size.

Python multiprocessing is a practical tool for CPU-bound parallelism, but it is not suited to all scales and scenarios. For datasets where the matrix is significantly larger, the per-worker data transmission cost would grow proportionally, potentially requiring a different approach such as memory-mapped files or shared memory. For very small datasets, the overhead dominates entirely and serial execution is faster. The 500-movie subset we used for evaluation sits in a range where 2-worker parallelism is beneficial but further scaling is not, which made it a particularly instructive test case for understanding these tradeoffs.

Overall, this project gave us practical experience with a parallelization challenge that reflects real engineering problems: the theoretical case for parallelism is clear, the implementation is correct, but the performance characteristics require empirical measurement and analysis to understand fully.

7.4 Future Work

- Use Python's multiprocessing.shared_memory module (introduced in Python 3.8) to give all workers read access to the matrix without copying it, which would eliminate the serialization bottleneck and allow more workers to be used effectively
- Evaluate on the full MovieLens 25M dataset where the computation dominates more strongly over overhead, which may shift the optimal worker count
- Compare performance against concurrent.futures.ProcessPoolExecutor and against a Dask-based implementation for distributed computing
- Add GPU-based similarity computation using CuPy to compare CPU multiprocessing against GPU parallelism
- Deploy the web application to a cloud server so the system is accessible at a public URL

8. Team Contributions

Team Member	Contributions
Yeshwanth Akula	Data preprocessing pipeline and matrix construction (preprocess.py), mean-centering normalization, serial baseline implementation (similarity_serial.py), FastAPI application setup and startup sequence (main.py), JWT authentication and password hashing (auth.py), SQLite database schema and activity logging (database.py), overall system integration and debugging across all modules
Harsha Reddy Erragunta	Parallel similarity implementation using multiprocessing.Pool (similarity_parallel.py), chunk distribution algorithm, worker function design and module-level placement, Windows spawn method debugging and resolution, cache management for parallel results, route handlers for all API endpoints (routes.py), request validation models (models.py)
Hemesh Phani Sai Bavirisetti	Performance evaluation script with pure NumPy parallel benchmark (evaluate.py), speedup graph generation using Matplotlib, SelfieSearch webcam capture and emotion detection (emotion.py), emotion-to-genre mapping logic, TMDB API integration for movie enrichment (tmdb.py), all three frontend dashboards with role-based UI (templates/), final report writing and documentation

9. References

[1] Goldberg, D., Nichols, D., Oki, B. M., & Terry, D. (1992). Using collaborative filtering to weave an information tapestry. Communications of the ACM, 35(12), 61–70. https://dl.acm.org/doi/10.1145/138859.138867

[2] Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. Proceedings of the 10th International Conference on World Wide Web, 285–295. https://dl.acm.org/doi/10.1145/371920.372071

[3] Harper, F. M., & Konstan, J. A. (2015). The MovieLens datasets: History and context. ACM Transactions on Interactive Intelligent Systems, 5(4), Article 19. https://dl.acm.org/doi/10.1145/2827872

[4] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. AFIPS Spring Joint Computer Conference, 483–485. https://dl.acm.org/doi/10.1145/1465482.1465560

[5] Python Software Foundation. (2024). multiprocessing — Process-based parallelism. Python 3.12 Documentation. https://docs.python.org/3/library/multiprocessing.html

[6] Serengil, S. I., & Ozpinar, A. (2020). LightFace: A hybrid deep face recognition framework. 2020 Innovations in Intelligent Systems and Applications Conference (ASYU), pp. 23–27. IEEE. https://ieeexplore.ieee.org/document/9259802

[7] The Movie Database. (2024). TMDB API Documentation. https://developer.themoviedb.org/docs/getting-started

[8] Tiangolo, S. (2024). FastAPI Documentation. https://fastapi.tiangolo.com/

[9] GroupLens Research. (2024). MovieLens Datasets. University of Minnesota. https://grouplens.org/datasets/movielens/

[10] Project source code and GitHub repository. https://github.com/Yesh-is-here2/movie-recommender